

JobsForWomen — Operations Runbook

Verified against source: 27 July 2026

Contents

JobsForWomen — Operations Runbook	1
0. Read this first (60-second orientation)	1
1. System topology	2
2. Environments	3
3. Environment variables	3
4. Deployment	4
5. Final deployment on the company domain	8
6. Final testing before handover	10
7. Pre-deploy checklist	11
8. Rollback	12
9. Incident triage	12
10. Repair procedures by subsystem	13
11. Rate limiting	17
12. Known defects and landmines	18
13. Local development	19
14. Load testing	20
15. Routine maintenance	20
16. Quick reference	21

JobsForWomen — Operations Runbook

Audience: any engineer who has never seen this codebase and has been handed a production incident. **Scope:** everything you need to understand the system, deploy it, diagnose a failure, and repair it. **Last verified against source:** 27 July 2026.

This document is self-contained. You do not need to read the other files in docs/ to use it — they provide topic-by-topic detail, this provides the operational picture. The rest of docs/ was audited and corrected against source on the same date; if the two ever diverge, trust this file.

0. Read this first (60-second orientation)

What it is	A job platform for women. Three roles: Candidate, Recruiter, Admin.
Backend	Express 5 + TypeScript on Node 22.x. Containerized; currently deployed on Render . Platform-neutral — see §4.
Frontend	React 19 + Vite SPA. Containerized; currently deployed on Vercel .
Database	PostgreSQL (currently Render), accessed via Prisma 6.19.3.
Cache/Queue	Redis (currently Upstash, <code>rediss://</code>). Used for rate limiting, permission cache, BullMQ jobs, and Socket.IO fanout.
Email	AWS SES v2, region <code>ap-south-1</code> , sending domain <code>mail.jobsforwomen.info</code> .

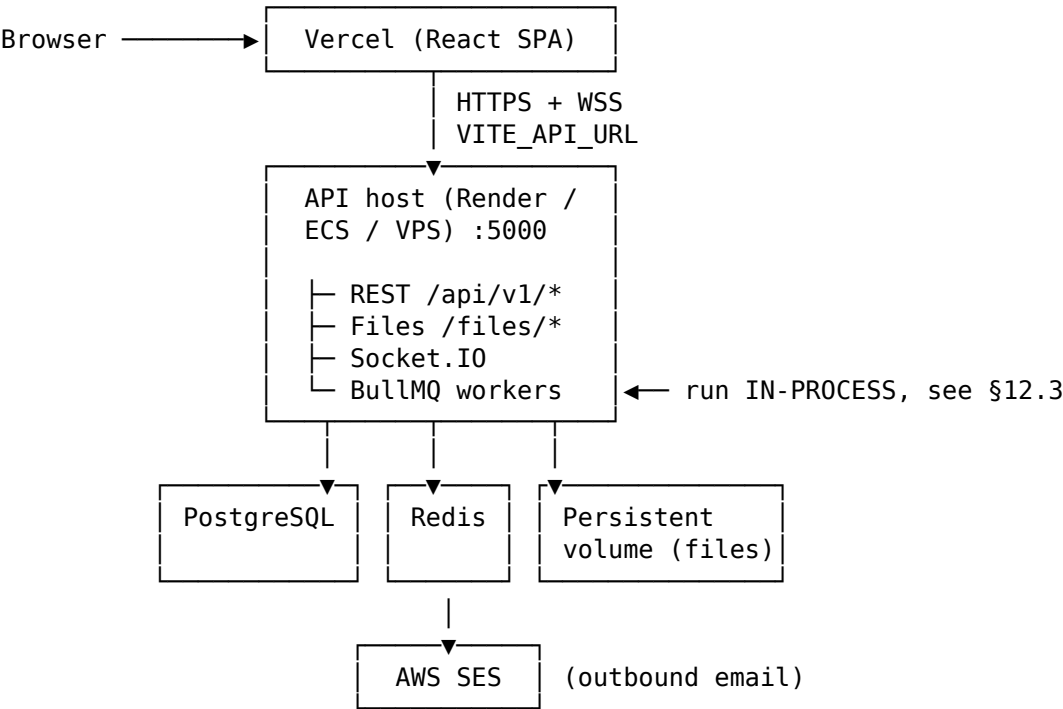
File storage	Local disk on a persistent volume (currently a Render Persistent Disk). Migrated off Cloudinary; no third-party storage remains. Socket.IO, namespaces /candidate, /recruiter, /admin.
Real-time	

First three commands in any incident:

```
curl https://<backend-host>/live      # process alive?
curl https://<backend-host>/ready    # process + database alive?
curl https://<backend-host>/health   # full subsystem breakdown (JSON)
```

/health is the single most useful endpoint in this system. It reports database status and latency, socket metrics, queue metrics, storage metrics, and email metrics in one response. Start there.

1. System topology



Everything runs in one Node process. The API, the Socket.IO server, and the BullMQ workers all share a single Node process. There is no separate worker service. See §12.3 — this matters during incidents.

Request path

- 1. helmet() — security headers
- 2. **Global rate limiter** — 100 req/60s per IP, applied to *every* route including /health
- 3. CORS allowlist check
- 4. requestId middleware — attaches correlation ID, visible in logs as [CID: ...]
- 5. Morgan HTTP logging
- 6. Body/cookie parsing
- 7. Route handler (with per-tier rate limiter + auth + RBAC as applicable)
- 8. errorHandler catch-all

Key source files

Concern	File
App wiring, CORS, health endpoints	backend/src/app.ts

Concern	File
Boot, Socket.IO attach, graceful shutdown	backend/src/server.ts
Env validation (Zod, fail-fast)	backend/src/shared/config/env.ts
Rate limiting (all tiers)	backend/src/shared/middleware/rateLimit.middleware.ts
File storage + URL signing	backend/src/shared/utils/fileStorage.ts
File serving route	backend/src/shared/routes/files.routes.ts
Queues, workers, dead-letter queue	backend/src/shared/queue/queue.ts
Cron schedules (not wired up — §12.1)	backend/src/shared/queue/scheduler.ts
Socket.IO server + Redis adapter	backend/src/shared/socket/socket.ts
Email (AWS SES)	backend/src/shared/utils/email.ts
SES bounce/complaint webhook	backend/src/shared/routes/sns.routes.ts
Auth logic	backend/src/modules/auth/auth.service.ts
Refresh-token cookie config	backend/src/shared/utils/cookies.ts
Frontend API client + token refresh	frontend/src/api/client.ts

2. Environments

	Backend	Frontend
Production (current)	Render web service	Vercel
Known deployed host	https://jobsforwomen-266w.onrender.com	https://jobs-for-women-ob43.vercel.app
Local	http://localhost:5000	http://localhost:5173

CORS allowlist is hardcoded in *two* places that must stay in sync: backend/src/app.ts and backend/src/shared/socket/socket.ts. Both allow http://localhost:5173, http://localhost:3000, https://jobs-for-women-ob43.vercel.app, plus whatever CLIENT_URL and FRONTEND_URL are set to. **If you add a new frontend domain, you must add it to both files or REST will work while WebSockets silently fail.**

3. Environment variables

Validated by Zod at boot in backend/src/shared/config/env.ts. **If validation fails the process exits immediately with code 1** and prints the offending fields. A crash-loop on a fresh deploy is very often a missing env var — check the first 20 lines of the log.

Required (no default — boot fails if missing)

Variable	Notes
DATABASE_URL	PostgreSQL connection string.
JWT_ACCESS_SECRET	Min 8 chars. Also signs file-download URLs — see §10.4 before rotating.
JWT_REFRESH_SECRET	Min 8 chars.
REDIS_URL	Upstash rediss:// URL.
SES_FROM	Sender identity on the SES-verified domain.
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_CALLBACK_URL	OAuth.

Has a default (safe to omit, but check the default is right for the environment)

Variable	Default	Notes
NODE_ENV	development	Must be production in any deployed environment — controls proxy trust, cookie security, and log level.
PORT	5000	
DISK_MOUNT_PATH	./uploads	Must point at persistent storage in production (Render Disk, AWS EFS, Docker volume).

Variable	Default	Notes
BACKEND_URL	http://localhost:5000	Must be the real public backend URL in production — used to build absolute file links.
JWT_ACCESS_EXPIRY	15m	
JWT_REFRESH_EXPIRY	7d	Candidates below this % cannot apply to jobs.
PROFILE_COMPLETION_THRESHOLD	70	
RATE_LIMIT_*	see §11	Seven tiers, each with _MAX and _WINDOW_MS.

Optional

CLIENT_URL, FRONTEND_URL (both feed the CORS allowlist), UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN, SUPPORT_EMAIL, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY (required on Render — no instance role), AWS_SES_REGION (default ap-south-1), SES_MAX_SEND_RATE_PER_SEC (default 1), SES_CONFIGURATION_SET, SNS_TOPIC_ARN.

Dangerous — never set in production

Variable	Effect
BYPASS_EMAIL_VERIFICATION=true	Lets accounts activate without verifying their email. Intended for local Docker and load-test seeding only. It is set in backend/docker-compose.yml (local only) and must never be added to a deployed environment.

Frontend

Variable	Notes
VITE_API_URL	Absolute backend URL. Inlined at build time — changing it requires a rebuild, not a restart. If missing in a production build, the client logs a critical console error and every API call fails.

4. Deployment

This section is platform-neutral. §4.1 states what *any* host must provide; §4.4–4.6 give concrete recipes for Render (the current deployment), AWS, and any Docker host. The current deployment is Render (backend) + Vercel (frontend), but nothing in the application is coupled to either.

4.1 What any platform must provide

Requirement	Why it matters
Node 22.x	Declared in backend/package.json engines; bcrypt compiles against it
PostgreSQL reachable via DATABASE_URL	Prisma 6.19.3
Redis reachable via REDIS_URL	BullMQ requires it. Without Redis no emails are sent — rate limiting and the permission cache degrade gracefully, queues do not
Persistent storage mounted at DISK_MOUNT_PATH	Container filesystems are wiped on redeploy. Without durable storage, every uploaded file is destroyed on the next release
A single instance	The persistent disk cannot be shared across replicas — the API cannot be horizontally scaled as built (§12.5)
WebSocket passthrough	Socket.IO needs upgrade support and a generous idle timeout on any proxy or load balancer
Outbound HTTPS	AWS SES (email) and Google (OAuth)
A way to run migrations	Shell, one-off task, or exec into the running container

Build and start commands, identical everywhere:

	Backend	Frontend
Build	npm run build (prisma generate && tsc)	npm run build (tsc -b && vite build)
Start Port	npm start (node dist/server.js) 5000 (PORT)	Serve dist/ statically n/a (static)

Two platform-independent traps:

- **The frontend needs an SPA rewrite** — all unmatched paths must return index.html, or refreshing /candidate/profile 404s. vercel.json handles this on Vercel; frontend/nginx.conf handles it in the container; on CloudFront it's a custom error response.
- **VITE_API_URL is inlined at build time.** It cannot be changed at runtime. Repointing the frontend at a different API requires a rebuild.

Migrations — run from anywhere that can reach the database:

```
npx prisma migrate deploy
```

Never run prisma db push --force-reset against production. It drops and recreates the schema, destroying all data. Take a database backup before any migration that drops or renames a column.

4.2 Containerization — backend

Both halves of the stack are containerized: backend/Dockerfile (this section) and frontend/Dockerfile (§4.3). Either can be deployed to Render with the service runtime set to Docker, to AWS ECS, to a VPS, or to any container platform.

backend/Dockerfile is a two-stage build and is production-ready. It is also what docker-compose.yml uses locally.

What it does:

Stage	Action
Builder	npm ci → npx prisma generate → npm run build (emits dist/)
Runtime	npm ci --only=production, copies dist/, prisma/, and node_modules/.prisma from the builder Creates /app/uploads, then chown -R node:node /app, then drops to USER node EXPOSE 5000, HEALTHCHECK polling /health every 30s, CMD node dist/server.js

Do not reorder the mkdir -p /app/uploads and chown lines. Docker copies a directory's existing ownership into a volume the first time that volume is mounted. If the directory doesn't exist (or isn't owned by node) before the volume mounts, the mount point ends up root-owned and the non-root container cannot write uploads — every upload fails with EACCES.

Known mismatch: the Dockerfile pins node:20-alpine, but backend/package.json declares "engines": { "node": "22.x" }. Containerized runs therefore execute on Node 20 while a Render Node-runtime deploy uses 22. Pick one — change the Dockerfile to node:22-alpine for consistency — and re-run the test suite after switching, since native modules (bcrypt) are rebuilt against the runtime.

Build and run standalone:

```
cd backend
docker build -t jobsforwomen-api:latest .

docker run -d --name jfw-api \
  -p 5000:5000 \
  --env-file .env \
  -e NODE_ENV=production \
  -e DISK_MOUNT_PATH=/app/uploads \
  -e BACKEND_URL=https://api.<your-domain> \
  -v jfw_uploads:/app/uploads \
  --restart unless-stopped \
```

```
jobsforwomen-api:latest
```

```
docker logs -f jfw-api          # expect: JobsForWomen API engine active on port 5000
docker exec jfw-api npx prisma migrate deploy
```

The `-v jfw_uploads:/app/uploads` volume is not optional. Without it, every uploaded file lives in the container’s writable layer and is destroyed when the container is replaced — which happens on every image update.

`backend/.dockerignore` excludes `node_modules`, `dist`, `logs/`, `uploads/`, and `.env` from the build context. Keep `.env` in it — without that exclusion, `COPY . .` bakes real database credentials and JWT secrets into an image layer, where they persist even if a later layer deletes the file.

4.3 Frontend container

`frontend/Dockerfile` builds the SPA and serves it through `nginx`. Three files make it work:

File	Purpose
<code>frontend/Dockerfile</code>	Two-stage build: Node 22 builder → <code>nginxinc/nginx-unprivileged:alpine</code> runtime, non-root, listening on 8080
<code>frontend/nginx.conf</code>	SPA fallback, gzip, cache headers, security headers, /healthz probe target
<code>frontend/.dockerignore</code>	Keeps <code>node_modules</code> , <code>dist</code> , <code>.env</code> , and <code>logs</code> out of the build context

```
cd frontend
docker build -t jobsforwomen-web:latest \
  --build-arg VITE_API_URL=https://api.jobsforwomen.info .

docker run -d --name jfw-web -p 8080:8080 --restart unless-stopped \
  jobsforwomen-web:latest
```

VITE_API_URL is a build argument, not a runtime variable. Vite substitutes `import.meta.env.*` into the bundle at compile time, so the backend URL is baked into the JavaScript. Passing it with `docker run -e` does nothing; pointing the frontend at a different API requires a rebuild. The Dockerfile fails the build outright if the argument is missing, rather than producing an image that looks fine and breaks on every request at runtime.

Two details in `nginx.conf` worth understanding before editing it:

- **index.html is served with no-cache, deliberately.** It references the content-hashed bundles in `/assets/`, which are cached for a year. If `index.html` is itself cached, returning visitors keep loading a previous deploy’s asset filenames — which 404 once those files are gone. This is the most common cause of “the site broke right after we deployed” on containerised SPAs.
- **The SPA fallback (`try_files $uri $uri/ /index.html`) is what makes deep links work.** It is the container equivalent of the rewrite in `vercel.json`. Remove it and refreshing `/candidate/dashboard` returns a 404.

The container listens on **8080**, not 80, because the unprivileged `nginx` base image runs as a non-root user and cannot bind ports below 1024. Map it to whatever the host or platform expects (`-p 80:8080`, or set the platform’s port to 8080).

A Content-Security-Policy is present but **commented out**. A working policy has to allow `connect-src` to both the API origin and its `wss://` scheme for Socket.IO, which is deployment-specific — an untested CSP would silently break real-time chat and every API call. Fill in the real origin and verify with the browser console open before enabling it.

Verification status: the frontend image has **not been built end-to-end** — no Docker daemon was available in the authoring environment. What was verified: `package-lock.json` exists (so `npm ci` will resolve), TypeScript compiles clean (`tsc --noEmit -p tsconfig.app.json`, which is the `tsc -b` half of `npm run build`), and vite build starts and enters transformation without configuration errors. `vite.config.ts` sets `no build.outDir/assetsDir` overrides, so Vite’s defaults (`dist/` and `dist/assets/`) apply

— which is what `nginx.conf` assumes. **Run docker build once locally before relying on this image.**

4.4 Recipe — Render (current deployment)

Render auto-deploys on push to main of <https://github.com/Rks052004/JobsForWomen.git>.

Backend — Web Service. Runtime can be Node (build `npm run build`, start `npm start`) or Docker (uses `backend/Dockerfile`).

1. Service → **Disks** → Add Disk. Note the mount path, e.g. `/var/data/uploads`.
2. Set `DISK_MOUNT_PATH` to exactly that path, `BACKEND_URL` to the service's public URL, and `NODE_ENV=production`.
3. Deploy, then open the **Shell** and run `npx prisma migrate deploy`.

Render-specific constraints:

- Persistent Disks require a **paid instance type** (Starter and up).
- **Single instance only** — a disk cannot be shared across replicas, so autoscaling is off the table until storage moves to S3.
- Disks are **not** reachable from Build or Pre-Deploy commands, nor from “One-off Jobs” (separate compute). Only the live instance's Shell can read or write them. Migrations are fine there; anything touching uploaded files is not.
- Free instances have no Shell at all, spin down after 15 minutes idle, and cold-start in 30–60s.

Frontend — Vercel. Build `npm run build`, output `dist`, set `VITE_API_URL`, redeploy after changing it. `vercel.json` already contains the SPA rewrite — do not remove it.

Managed services: Render PostgreSQL; Redis from Upstash (`rediss://`) or Render Key Value.

4.5 Recipe — AWS

The same architecture maps onto AWS as follows. Persistent storage is the only genuinely constrained choice.

Concern	Service	Notes
API	ECS Fargate (or App Runner, or EC2 + Docker)	Runs <code>backend/Dockerfile</code> . Desired count = 1 until storage moves to S3
Database	RDS for PostgreSQL	<code>DATABASE_URL</code> in Secrets Manager, injected as a task secret
Redis	ElastiCache for Redis (or Upstash)	Must be reachable from the task's security group
File storage	EFS , mounted at <code>DISK_MOUNT_PATH</code>	The only option that keeps the code unchanged <i>and</i> survives task replacement. EBS works for a single EC2 host but not Fargate; instance storage does not persist
Frontend	S3 + CloudFront	Set a custom error response mapping <code>403/404</code> → <code>/index.html</code> (200) for the SPA rewrite. Or run <code>frontend/Dockerfile</code> on ECS behind the same ALB
TLS / routing	ALB + ACM	Enable WebSocket support and stickiness , and raise the idle timeout — the 60s default will drop Socket.IO connections
DNS	Route 53	<code>api.<domain></code> → ALB; <code><domain></code> → CloudFront
Secrets	Secrets Manager / SSM	Never bake <code>.env</code> into the image; <code>backend/.dockerignore</code> excludes it for this reason
Logs	CloudWatch Logs	The app writes structured logs to <code>stdout/stderr</code>
Migrations	One-off ECS task, same image and env, command overridden to <code>npx prisma migrate deploy</code>	

AWS-specific traps:

- **app.set("trust proxy", 1) assumes exactly one proxy hop.** Behind an ALB alone that is correct. Put CloudFront in front of the API as well and req.ip resolves to the wrong address, which makes every IP-keyed rate-limit tier bucket all traffic into one counter. Match the hop count to your real chain — but never set it to true, which lets clients forge X-Forwarded-For and evade rate limiting entirely.
- **ALB idle timeout** must exceed the WebSocket keepalive or chat drops silently.
- **EFS burst credits** are worth watching if upload volume grows; bursting throughput is fine at current scale.
- **Moving uploads to S3** is what unlocks multi-task scaling. The change is contained: shared/utils/fileStorage.ts is the only module touching the filesystem, and its HMAC signed-URL layer maps cleanly onto S3 presigned URLs.

4.6 Recipe — any Docker host or VPS

1. Provision PostgreSQL and Redis (managed, or as additional containers).
2. Run both images from §4.2 and §4.3, with a named volume or bind mount for uploads.
3. Terminate TLS and reverse-proxy with nginx. backend/nginx.conf is a working starting point: HTTP→HTTPS redirect, TLS 1.2/1.3, HSTS, gzip, client_max_body_size 10M (matching Multer's cap), and a /socket.io/ location with upgrade headers and 86400s timeouts.
4. Issue certificates with Certbot — the config already includes the /.well-known/acme-challenge/ location. **Verify the renewal timer is active** (systemctl list-timers | grep certbot); an expired certificate is a total outage.
5. docker exec into the API container and run npm run prisma migrate deploy.

The CSP in backend/nginx.conf sets script-src 'none'; style-src 'none', which blocks the Swagger UI at /api/v1/api-docs from rendering. Relax it for that path or accept that the docs page will not load.

5. Final deployment on the company domain

The current deployment uses platform-provided hostnames (*.onrender.com, *.vercel.app). Moving to the company's own domain touches seven places. Missing any one of them produces a partially-working site, which is harder to debug than a clean failure.

The repo already anticipates this layout — backend/nginx.conf and backend/src/shared/utils/swagger.ts both reference api.jobsforwomen.info:

Host	Serves
jobsforwomen.info (+ www)	Frontend SPA
api.jobsforwomen.info	Backend API, file delivery, Socket.IO

5.1 Cutover checklist

Work in this order. Steps 2–6 can be done before DNS moves, so the switch itself is low-risk.

1. DNS

Record	Points at
jobsforwomen.info → A / ALIAS	Vercel (or your static host)
www → CNAME	Vercel
api → CNAME (Render) or A (VPS)	Backend host

Allow for propagation before testing; a stale resolver cache looks exactly like an outage.

2. TLS

- *Render / Vercel*: add the custom domain in the dashboard; certificates are issued and renewed automatically.

- **VPS:** backend/nginx.conf is already written for Let's Encrypt, including the /.well-known/acme-challenge/ location and an HTTP→HTTPS 301. Issue the certificate with `certbot certonly --webroot -w /var/www/certbot -d api.jobsforwomen.info`, then confirm the renewal timer is active (`systemctl list-timers | grep certbot`). An expired certificate is a total outage.

3. Backend environment variables (Render dashboard, or the container's --env-file)

Variable	Set to
BACKEND_URL	https://api.jobsforwomen.info
CLIENT_URL	https://jobsforwomen.info
FRONTEND_URL	https://jobsforwomen.info
GOOGLE_CALLBACK_URL	https://api.jobsforwomen.info/api/v1/auth/google/callback
NODE_ENV	production

BACKEND_URL is what every stored file URL is built from. Getting it wrong means uploads succeed but the links in API responses point at the old host.

4. CORS allowlist — two files, both required

Add the new frontend origin to the hardcoded allowlist in **both**:

- backend/src/app.ts
- backend/src/shared/socket/socket.ts

Setting CLIENT_URL / FRONTEND_URL covers this without a code change, since both files fold those env vars into the allowlist. But if you hardcode instead, doing only one file gives you a site where REST works and real-time chat silently fails — the single most common symptom of a half-finished domain migration.

5. Google OAuth — in Google Cloud Console, add the new callback URI to *Authorized redirect URIs* and the new frontend origin to *Authorized JavaScript origins*. Both must match exactly, including scheme and absence of a trailing slash.

6. Email sender domain — mail.jobsforwomen.info is already verified in AWS SES in ap-south-1; set SES_FROM to an address on it (e.g. noreply@mail.jobsforwomen.info). Two things to confirm at cutover: the account is out of the **sandbox** (until then only individually verified recipients receive mail — \$10.5), and DKIM/SPF records for the sending domain remain published in DNS if you move nameservers. Note that email.ts falls back to https://jobsforwomen.info when FRONTEND_URL and CLIENT_URL are both unset, so links in emails may look correct while the app is still on the old host — set the env vars explicitly rather than relying on that fallback.

7. Frontend — set VITE_API_URL=https://api.jobsforwomen.info and **rededeploy**. Vite inlines this at build time; changing the variable without a rebuild has no effect. On Vercel that means setting the environment variable and triggering a new deployment; if serving the container instead (\$4.3), rebuild the image with --build-arg VITE_API_URL=https://api.jobsforwomen.info.

5.2 VPS-specific notes

If the company hosts the API themselves rather than on Render, backend/nginx.conf is a working reverse-proxy config: HTTP→HTTPS redirect, TLS 1.2/1.3, HSTS, gzip, client_max_body_size 10M (matching Multer's cap), a /socket.io/ location with WebSocket upgrade headers and 86400s timeouts, and proxying to localhost:5000.

Two things to fix before using it as-is:

- **Its CSP will break the API docs.** The header sets script-src 'none'; style-src 'none', which blocks the Swagger UI at /api/v1/api-docs from loading. Either relax the CSP for that path or accept that the docs page won't render in production.
- **app.set("trust proxy", 1)** is already correct for exactly one proxy hop. If you add a second layer (Cloudflare in front of nginx, say), req.ip will resolve to the intermediate proxy and every IP-keyed rate limit tier will bucket all traffic together. Adjust the hop count to match your actual chain — but never set it to true, which lets clients forge X-Forwarded-For and evade rate limiting entirely.

5.3 Post-cutover smoke test

Run all of these against the new domain before declaring the migration done:

```
D=https://api.jobsforwomen.info
curl -s $D/live # Live
curl -s $D/ready # Ready
curl -s $D/health | jq '.data.database, .data.storage'
curl -sI $D/health | grep -i x-ratelimit # limiter active
curl -sI https://jobsforwomen.info | grep -i "^HTTP" # 200
```

Then, in a browser on the new frontend domain: log in, confirm no CORS errors in the console, open a conversation and send a message (proves Socket.IO + WSS upgrade through the proxy), upload a resume and reopen it (proves storage write + signed URL under the new BACKEND_URL), and trigger a password reset to confirm mail delivery from the verified domain.

6. Final testing before handover

This system has not yet had a full end-to-end acceptance pass on production infrastructure. Compile checks and the unit suite pass, and rate limiting has been load-tested, but the items below have not been verified end-to-end against a real deployment with a real domain. Do not treat the platform as accepted until they are.

Track each row as pass/fail with the date and who ran it. Anything failing is a release blocker, not a footnote.

6.1 Automated checks (fast, run first)

Check	Command	Pass condition
Backend types	cd backend && npx tsc --noEmit	No errors
Frontend types	cd frontend && npx tsc --noEmit	No errors
Unit/integration tests	-p tsconfig.app.json cd backend && npm test	Suite green
Migration state	npx prisma migrate status	No drift, no pending
Rate-limit tiers	k6-load-tests/ suite, one script at a time (\$14)	checks > 95%, X-RateLimit-Limit matches each tier
Email provider	npm run email:test <addr>	SES accepted email id=...

6.2 Manual functional pass

Run each role's full journey on the deployed environment, not locally.

Candidate: register → verify email → complete profile past the 70% threshold → browse and filter jobs → save a job → apply → upload a resume → reopen the uploaded resume → message a recruiter → receive a reply → check notifications → reset password.

Recruiter: register → company verification submitted → (admin approves) → log in → post a job → (admin approves job) → view applicants → download a resume → shortlist → schedule an interview → release an offer with an attached offer letter → message a candidate → invite a team member and have them accept.

Admin: log in → approve/reject a company → approve/reject a job → approve a perk request → view System Health (every subsystem green) → view activity logs → manage users → create another admin.

6.3 Cross-cutting checks

Area	What to verify
Real-time	Two browsers, two accounts, same conversation — messages arrive without refresh. Unread badges increment on the <i>inactive</i> conversation

Area	What to verify
File lifecycle	Upload, reopen after >1 hour (link should 403 and recover on page reload), replace, delete. Confirm files survive a redeploy — this is the persistent-disk test and the highest-consequence one
Email	Every template actually arrives from the verified domain and its links point at the production frontend, not localhost
Auth edges	Expired access token auto-refreshes without a visible error; blocked/suspended accounts are refused with the right message; recruiter with an unapproved company cannot log in
Responsive	Every page at mobile, tablet, and desktop widths
Error visibility	Kill the backend mid-session — the UI must show a real error, not hang on a skeleton
Loading states	Every dashboard and list page shows a skeleton, never a blank screen or raw “Loading...”

6.4 Security items that must be closed before handover

Item	Action
Seeded default admin	prisma/seed creates admin@jobsforwomen.info with password admin123 , status Active. Change this password or delete the account before the site is publicly reachable. It is a documented credential in docs/03_INSTALLATION_GUIDE.md and docs/05_DATABASE.md
JWT secrets	Confirm both are long random values, not the development placeholders
BYPASS_EMAIL_VERIFICATION	Confirm unset in production
Rate limits	Confirm the tier ceilings suit expected real traffic (§11)
Uploads	Confirm oversized and wrong-type files are rejected. Note there is no malware scanning (§12.7) — the company should know this
HTTPS	No mixed-content warnings; HSTS present
Swagger	Decide whether /api/v1/api-docs should be publicly reachable in production. It is currently unauthenticated

6.5 Known gaps to disclose

Do not let these surface as surprises after handover. Each is documented in §12:

- Scheduled/cron jobs never run — no digest emails, no auto-close of expired jobs (§12.1)
- Three queues have no workers; jobs sent to them accumulate silently (§12.2)
- Single-instance only; cannot horizontally scale while files are on a persistent disk (§12.5)
- Images stored unoptimized (§12.4)
- No malware scanning on uploads (§12.7)

7. Pre-deploy checklist

1. cd backend && npx tsc --noEmit — clean.
2. cd frontend && npx tsc --noEmit -p tsconfig.app.json — clean.
3. npx prisma migrate status — no unexpected drift.
4. Confirm JWT_ACCESS_SECRET / JWT_REFRESH_SECRET are not the development placeholder values.
5. Confirm BYPASS_EMAIL_VERIFICATION is **not** set on Render.
6. Confirm Google OAuth authorized redirect URI matches GOOGLE_CALLBACK_URL exactly.
7. Take a database backup if the deploy includes a migration.

8. Rollback

Code rollback

Preferred — safe, keeps history, no force-push:

```
git revert <bad-commit-sha>
git push origin main          # Render auto-deploys the revert
```

Only if you must erase history (solo repo, nothing depends on the commit):

```
git reset --hard <last-good-sha>
git push --force origin main
```

Render also supports redeploying a previous build directly from the dashboard (service → Events → Redeploy), which is faster than a git round-trip when you need the bleeding to stop immediately.

Database rollback

Prisma has no automatic down-migration. Restore from a Render Postgres backup (dashboard → database → Backups) and then re-apply migrations forward. **Take a manual backup before any migration that drops or renames a column.**

What rollback does *not* undo

- Files already written to the Persistent Disk stay there.
- Emails already dispatched to SES cannot be recalled.
- Rate-limit counters in Redis persist until their window expires.

9. Incident triage

Work top to bottom. Stop at the first check that fails and jump to the linked section.

1. `curl /live`
 - └ no response / timeout → §10.1 API down
 - └ "Live" → continue
2. `curl /ready`
 - └ 503 → §10.2 Database
 - └ "Ready" → continue
3. `curl /health` (read the JSON)
 - └ `database.latencyMs` high → §10.2 Database
 - └ `queues.failedJobs` climbing → §10.6 Queues
 - └ all nominal → continue
4. What is the user-visible symptom?
 - └ "can't log in" → §10.7 Auth
 - └ "429 / too many requests" → §11 Rate limiting
 - └ "file won't open / 403" → §10.4 File storage
 - └ "no emails arriving" → §10.5 Email
 - └ "chat/notifications dead" → §10.3 Redis, then §10.8 Sockets
 - └ "blank page / white screen" → §10.9 Frontend
 - └ "everything is slow" → §10.1, then §10.2

Where to find logs: Render dashboard → service → Logs (live tail). Locally, `backend/logs/error.log` and `backend/logs/combined.log`. Every log line carries a correlation ID as `[CID: <id>]` — `grep` by that to follow a single request end-to-end.

10. Repair procedures by subsystem

10.1 API down or crash-looping

Symptoms: /live times out; Render shows repeated restarts.

Cause	How to confirm	Fix
Missing/invalid env var	Log shows [FAIL] Invalid environment configuration: then a Zod field dump, then exit	Add the missing variable in Render → Environment; redeploy
Uncaught exception	Log shows Uncaught Exception thrown: + stack. server.ts calls process.exit(1) on these by design	Read the stack, fix or revert the offending commit (\$8)
Cold start (free tier only)	First request after idle takes 30–60s then succeeds	Not a fault. Upgrade off free tier to eliminate
Port already bound	EADDRINUSE	Only a local problem; kill the stale process

The process handles SIGTERM/SIGINT gracefully (server.close() then exit 0), so a normal Render restart should never truncate in-flight requests.

10.2 Database

Symptoms: /ready returns 503; /health shows database.status: "DOWN"; API returns 500s broadly.

1. Check Render → database → status and connection count. Render Postgres has a connection cap; exhausting it presents as intermittent failures rather than a clean outage.

2. Check for long-running queries:

```
SELECT pid, state, query_start, query
FROM pg_stat_activity
WHERE state = 'active'
ORDER BY query_start;
```

Terminate a stuck one with SELECT pg_terminate_backend(<pid>;).

3. Check migration drift: npx prisma migrate status from the Render Shell.
4. If Prisma Client is out of sync with the schema (errors like “*property does not exist on type*” at build, or unexpected column errors at runtime), run npx prisma generate and redeploy.

Always run Prisma commands from inside backend/. Running them at the repo root prompts to install Prisma v7 and will not find the schema.

10.3 Redis

Symptoms: rate limiting behaves per-instance; permission cache misses; Socket.IO broadcasts don’t cross instances; log shows Redis connection error: ... Running without permission cache.

The system degrades gracefully rather than failing. Understand what silently changes when Redis is down:

Subsystem	Behaviour without Redis
Rate limiting	Falls back to an in-process Map. Limits still enforced, but per instance , not globally.
Permission cache	Bypassed; permissions read from DB every time (slower, still correct).
Socket.IO	Falls back to the in-memory adapter — no cross-instance fanout . Logged as [SocketIO] Redis client unavailable.
BullMQ queues	Do not work at all . No jobs are enqueued or processed, so no emails are sent.

So: “*emails stopped but the site works*” is a classic Redis-down signature.

Fix: verify the Upstash instance is up and REDIS_URL is correct (note the rediss:// scheme — double s, TLS). Restart the Render service after correcting it; the Redis client and Socket.IO adapter are only wired at boot.

10.4 File storage

Files live at DISK_MOUNT_PATH/jfw/<type>/<filename>, where <type> is one of: logos, gallery, resumes, offer-letters, perk-documents, company-verification.

Access model:

- logos and gallery are **public** — served with no signature (they appear on public job listings).
- Everything else is **private** — requires a valid ?exp=<unix>&sig=<hmac> query string. URLs are signed automatically by sendSuccess() in response.ts for every /files/jfw/ URL anywhere in any API response. TTL is **1 hour**.

Symptom	Cause	Fix
403 “This link has expired or is invalid”	Signature older than 1 hour, or the page was left open	Reload the page to get freshly-signed URLs. Working as designed.
All private files 403 immediately after a deploy	JWT_ACCESS_SECRET changed — it is the file-signing secret	Expected and self-healing: links regenerate on next page load. See the rotation warning below.
404 for files that used to exist	Persistent Disk not mounted, or DISK_MOUNT_PATH wrong — files were on ephemeral disk and got wiped	Fix the mount (§4.1). Lost files are unrecoverable ; users must re-upload.
Upload fails with EACCES	Disk mount owned by root; container runs as non-root node	The Dockerfile creates /app/uploads before chown -R node:node /app specifically to prevent this. If you change the Dockerfile, preserve that ordering.
Upload rejected as invalid type	Magic-byte check failed — file content doesn’t match declared MIME type	Working as designed (scanFileForVirus in fileStorage.ts). Note this is structural validation only, not malware scanning .

Rotating JWT_ACCESS_SECRET has two effects, not one: it logs out every user *and* invalidates every outstanding file-download link. The second is usually forgotten. Both recover on the next page load, but expect a burst of 403s in the logs.

Health check: verifyStorageConnection() writes and deletes a probe file, so a green storage status in the admin System Health page means the disk is genuinely writable, not just configured.

Orphan audit: runOrphanAssetCleanup() cross-references DB rows against files on disk and reports orphans and missing-referenced files. It is **read-only and never deletes anything** — deletion is a deliberate human action.

10.5 Email

Everything funnels through one chokepoint: EmailService.sendMail() in backend/src/shared/utils/email which calls the **AWS SES v2 API**.

Flow: EventBus.publish → email listener → addJob("email", ...) → BullMQ (rate-limited) → worker → EmailService → SES.

Current SES state — read this before diagnosing anything:

Region	ap-south-1 (identities are regional — a domain verified elsewhere does not exist here)
Verified sending domain	mail.jobsforwomen.info
Account status	Sandbox (production access applied for)
Send rate	1 email/second
Daily cap	240 emails per 24 hours
Deliverable recipients	Only individually verified addresses

Sandbox is a recipient restriction, not a sender one. Verifying the sending domain does not lift it. While sandboxed, mail to anyone unverified fails with `MessageRejected: Email address is not verified` — which the app classifies as **permanent**, so the job fails once instead of retrying three times and burning three of the 240 daily sends.

Symptom	Cause	Fix
No emails at all, site otherwise healthy	Redis down → BullMQ dead	\$10.3
MessageRejected: Email address is not verified	Sandbox recipient restriction	Verify the recipient in the SES console, or wait for production access
TooManyRequestsException	Exceeded 1 send/sec	Worker is limited to <code>SES_MAX_SEND_RATE_PER_SEC</code> (default 1) with concurrency: 1. Lower it if this recurs; raise it once production access grants a higher rate
Sends work early in the day, fail later	240/24h quota exhausted	<code>npm run email:test</code> prints <code>SentLast24Hours</code> vs <code>Max24HourSend</code>
CredentialsProviderError	No credentials resolved	Set <code>AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY</code> — Render has no instance role
MessageRejected about the identity, despite a verified domain	Wrong region	Confirm <code>AWS_SES_REGION</code> matches where the domain was verified
Emails skipped, log says <code>email_automation</code> feature flag is disabled	Admin turned off the <code>email_automation</code> feature flag	Re-enable in the admin panel. <code>sendWelcome</code> and <code>sendPasswordReset</code> are exempt and always send — deliberately, so nobody gets locked out
Job failed once and went straight to the DLQ	A permanent rejection, converted to BullMQ <code>UnrecoverableError</code>	Expected behaviour — see §10.6

Test the transport without a full signup flow:

```
cd backend
npm run email:test <recipient@example.com>
```

It prints region, sender, and credential source, then calls SES `GetAccount` — which reports production-access status and remaining quota **without consuming any** — before sending. Success prints `[EmailService] SES accepted email id=<MessageId>`.

Bounces and complaints. SES publishes these via SNS to `POST /api/v1/emails/sns` (shared/routes/sns.routes.ts), which handles the SNS subscription confirmation handshake automatically and writes `EMAIL_BOUNCED_PERMANENT` / `EMAIL_BOUNCED_TRANSIENT` / `EMAIL_COMPLAINT` rows to `AuditLog`. This requires `SES_CONFIGURATION_SET` to be set — without a configuration set SES publishes no events at all and the endpoint stays silent. The older generic `/api/v1/emails/bounce` and `/complaint` routes remain for manual testing but receive nothing from SES.

Watch the complaint rate. Sustained complaints above ~0.1% put SES sending privileges at risk, and AWS reviews this when granting production access.

Digests versus the daily cap. The daily/weekly digest jobs enqueue **one email per candidate**. At any real user count that alone exceeds a 240/day sandbox quota. Those jobs do not currently run (the scheduler is never wired up — §12.1), but they will the moment it is. Add batching or a send budget before enabling them.

10.6 Queues and the dead-letter queue

Queues declared: email, notifications, audit, cleanup, reports, plus dead-letter.

Workers are registered for email and cleanup only. See §12.2 for what that means.

- Retry policy: **3 attempts**, exponential backoff starting at 5s.
- On final failure the job is pushed to the dead-letter queue with its original queue name, job name, sanitized payload, failure reason, and attempt count — and an `AuditLog` row with action `QUEUE_JOB_FAILED_DLQ` is written.

- Payloads are **sanitized before storage**: any key matching password|token|secret|apikey|authorization is replaced with [REDACTED], so raw reset tokens never sit in Redis or the audit log.
- Nothing ever consumes dead-letter. It is a durable inspection area by design, which is also what makes recursive DLQ handling impossible.

To inspect: the admin System Health page surfaces the pending dead-letter count via `getDeadLetterQueueStats()`. For detail, query the `AuditLog` table for `action = 'QUEUE_JOB_FAILED_DLQ'` — that carries the failure reason and sanitized payload.

To replay: there is no built-in replay. Read the DLQ entry, fix the root cause, then re-trigger the originating action (e.g. have the user request another password reset).

10.7 Authentication

Token model — get this right or you will chase ghosts:

- **Access token:** JWT, 15-minute expiry, sent in the `Authorization: Bearer <token>` header. Stored in `localStorage` as `jwt_token`. **Not a cookie.**
- **Refresh token:** `httpOnly` cookie named `jid`, **path-scoped to `/api/v1/auth/refresh` only**. In production it is `Secure + SameSite=None`; locally `SameSite=Lax`.

Because the refresh cookie is path-scoped, it is *not* sent to any other endpoint. Anything that assumes cookie auth works across the API is wrong.

The frontend does **single-flight refresh**: the first 401 triggers one `POST /auth/refresh`; concurrent callers await that same promise and retry once each.

Symptom	Cause	Fix
Infinite redirect to <code>/auth/login</code>	Refresh cookie missing/rejected — usually <code>SameSite=None</code> without <code>Secure</code> , or a domain mismatch	Confirm <code>NODE_ENV=production</code> on Render (it drives the cookie flags) and that the frontend origin is in the CORS allowlist in both <code>app.ts</code> and <code>socket.ts</code>
“Please verify your email address first”	Account still <code>PendingVerification</code>	Have the user click the verification link, or set status directly: <code>UPDATE "User" SET status = 'Active' WHERE email = '...'</code>
Recruiter can’t log in	Company not yet approved — checked in <code>assertRecruiterCompanyApproved()</code>	Approve the company in the admin panel. The error message states which state it’s in (<code>rejected</code> , <code>info_requested</code> , or <code>pending</code>)
Google OAuth callback error	<code>GOOGLE_CALLBACK_URL</code> doesn’t exactly match the URI registered in Google Cloud Console	Align them, including scheme and trailing path
Everyone logged out at once	<code>JWT_ACCESS_SECRET</code> or <code>JWT_REFRESH_SECRET</code> was rotated	Expected. Also invalidates file links (§10.4)

User statuses: `PendingVerification`, `PendingApproval`, `Active`, `Rejected`, `Suspended`, `Blocked`. Login rejects all but `Active` (and `PendingVerification` only when `BYPASS_EMAIL_VERIFICATION=true`).

10.8 Sockets / real-time

Socket.IO is attached to the HTTP server in `server.ts` via `initSocket(server)`. Namespaces: `/candidate`, `/recruiter`, `/admin`. Handshake requires a valid access token (`socket.handshake.auth.token`).

`connectionStateRecovery` is enabled with a 2-minute window, so brief network drops recover without losing events.

Symptom	Check
Chat/notifications dead for everyone	Is the process up (§10.1)? Is the token valid — an expired JWT fails the handshake
Works for some users, not others	Redis adapter not attached → no cross-instance fanout. Log shows [SocketIO] Failed to attach Redis adapter or Redis client unavailable. Currently single-instance so this is latent, but it becomes a live bug the moment a second instance exists

Symptom	Check
Handshake rejected from a new domain	CORS allowlist in <code>socket.ts</code> — separate from <code>app.ts</code> (§2)

10.9 Frontend

Symptom	Cause	Fix
White screen, console: VITE_API_URL is missing 404 on refreshing a nested route Blank flash between route changes	Env var absent from the Vercel production build SPA rewrite missing Route chunk loading without a Suspense fallback	Add it in Vercel → Settings → Environment Variables, redeploy Restore <code>vercel.json</code> (§4.1) All three route routers now wrap their route tree in <code><Suspense></code> with a skeleton fallback. If you add a new lazy router, do the same
Page stuck on a loading skeleton forever	The underlying fetch failed but <code>isLoading</code> was never cleared	Check the finally block sets <code>setIsLoading(false)</code> ; check the Network tab for the failed call
All API calls fail with CORS errors	Frontend origin not in the backend allowlist	§2

11. Rate limiting

Implemented in `backend/src/shared/middleware/rateLimit.middleware.ts`. Redis-backed via atomic INCR/EXPIRE, with automatic fallback to an in-process Map if Redis is unavailable. Bypassed entirely when `NODE_ENV=test`.

Every response carries `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`. A rejection is 429 with `Retry-After`.

Tier	Default limit	Keyed by	Applied to
Global	100 / 60s	IP	Every route, before all others
Auth	10 / 15 min	IP	login, register, oauth, refresh, forgot/reset-password, verify-email, invitation accept
Admin	200 / 60s	User ID	Entire admin route group
Recruiter	150 / 60s	User ID	Entire recruiter route group
Candidate	150 / 60s	User ID	Entire candidate route group
Upload	10 / 10 min	User ID	Resume, logo, gallery, verification and perk document uploads
Search	300 / 60s	IP	Job search/browse/recommendations (layered over the candidate tier)

All limits are overridable per environment via `RATE_LIMIT_<TIER>_MAX` and `RATE_LIMIT_<TIER>_WINDOW_MS` — no code change or redeploy needed to retune.

User-ID keying is deliberate for authenticated tiers: keying those by IP would let one user behind a shared or NAT'd address (an office, a mobile carrier) degrade service for everyone on it, and would let a single abuser evade the limit by roaming IPs.

The single most common diagnostic mistake: the global tier (100/60s) is *stricter* than most others and runs first. A client firing 150 requests to walk the candidate tier's budget will be rejected at ~100 by the **global** limiter, not the candidate one. Check `X-RateLimit-Limit` on the 429 to see which tier actually rejected you. Pace requests below 100/min to reach any higher tier's boundary.

Legitimate users hitting 429s? Raise the relevant `RATE_LIMIT_*_MAX` in Render's environment settings and restart. Check first whether it's really the global tier that's biting.

12. Known defects and landmines

These are real, currently-true properties of the deployed system. They are the things most likely to waste your time.

12.1 Scheduled/cron jobs never run

`bootstrapScheduler()` in `backend/src/shared/queue/scheduler.ts` defines five repeatable jobs — expired-invitation cleanup (hourly), job-expiry closure (daily), old-notification cleanup (weekly), and daily/weekly digest compilation — plus a monthly admin report.

It is never called from anywhere in the codebase. Verified by `grep`: the only occurrences of the symbol are its own definition and export.

Consequences: expired invitations are never cleaned up; jobs past their deadline are never auto-closed; read notifications older than 30 days accumulate forever; digest emails have never been sent.

Fix, if these are wanted: import and call it in `server.ts` after `initSocket(server)`. Before doing so, confirm the cleanup logic is correct against current data volumes — `jobExpiry` loads every approved and pending job into memory and iterates, which is fine at current scale but is an obvious future bottleneck.

12.2 Three queues have no workers

`queueNames` declares `email`, `notifications`, `audit`, `cleanup`, `reports`. Only `email` and `cleanup` get a `registerWorker()` call.

Any job added to `notifications`, `audit`, or `reports` is enqueued into Redis and **never processed** — it sits in the waiting state indefinitely, consuming Redis memory. It does not fail, so it never reaches the dead-letter queue and never appears in failure metrics. Silent accumulation.

Check for this with BullMQ's job counts on those queues if Redis memory grows unexpectedly.

12.3 Workers run in the API process

`queue.ts` registers its workers as a module side-effect at import time, and `app.ts` imports it (for `queueMetrics` on the health endpoint). So workers start whenever the API starts.

Implications: heavy job processing competes with request handling for the same event loop; restarting the API also restarts the workers; and there is no separate worker service to scale or monitor independently. `docs/16_DEPLOYMENT.md` describes a separate Background Worker Service — that is not how it is deployed.

12.4 Images are stored unoptimized

Cloudinary used to resize and compress public images (800×800 cap, auto quality). The local-disk replacement does not — `sharp` was not available when the migration was written. Multer still caps logo uploads at 2MB, so disk growth is bounded but larger than before. To restore optimization: `npm install sharp` and reintroduce a resize step in `uploadFile()` in `fileStorage.ts` (the exact insertion point is marked with a comment).

12.5 File uploads block horizontal scaling

Restated because it constrains every future infrastructure decision: a Render Persistent Disk cannot be shared between instances. The service is single-instance until file storage moves to object storage.

12.6 Documentation currency

The docs/ set was audited and corrected on 27 July 2026: every Cloudinary reference was replaced with the local-disk model, the dangerous `prisma db push --force-reset` command was removed from `16_DEPLOYMENT.md`, and missing environment variables (`SMTP_HOST`/`SMTP_USER`, all seven `RATE_LIMIT_*` tiers, `DISK_MOUNT_PATH`, `BACKEND_URL`) were documented.

This runbook remains the authoritative operational reference; the numbered docs are the topic-by-topic detail. If they ever diverge again, trust this file and correct the other.

12.7 Malware scanning is not implemented

`scanFileForVirus()` performs magic-byte structural validation only — it confirms file content matches the declared MIME type, catching a renamed executable. It is **not** antivirus. No ClamAV/VirusTotal-style scanning is wired in. The function's own log line says so explicitly. Do not represent this as malware protection.

13. Local development

13.1 Without Docker

```
# Backend
cd backend
npm install
npx prisma generate
npx prisma migrate deploy
npm run dev                # ts-node-dev, hot reload, :5000

# Frontend
cd frontend
npm install
npm run dev                # Vite, :5173
```

Requires local (or remote) PostgreSQL and Redis reachable via `DATABASE_URL` / `REDIS_URL`.

13.2 With Docker (self-contained)

```
cd backend
docker compose up --build
```

Brings up three containers: `api` (:5000), `db` (Postgres 15), `redis` (Redis 7). The compose file **overrides** `DATABASE_URL` and `REDIS_URL` to point at the local containers, so it never touches production data.

Then apply migrations:

```
docker compose exec api npx prisma migrate deploy
```

Persistence test: uploads are stored in the named volume `uploads_data`. `docker compose down` preserves it; `docker compose down -v` deletes it. To verify persistence works, upload a file, `docker compose down`, `docker compose up`, and confirm the file is still served.

Known Windows issue: the `db` and `redis` services intentionally have **no host port mappings**. Windows was refusing to bind *any* host port ("An attempt was made to access a socket in a way forbidden by its access permissions" on both 5432 and 5433), which indicates a Windows-level restriction — a stuck winnat service or a Hyper-V dynamic port exclusion range — rather than a genuine port conflict. The `api` container reaches both over Docker's internal network by service name regardless, so removing the mappings sidesteps it entirely. If you need to attach a GUI client (`psql`, `DBeaver`, `TablePlus`) from the host, add a `127.0.0.1:<port>:5432` mapping back after fixing the underlying issue with, as Administrator:

```
net stop winnat
net start winnat
```

13.3 Creating an admin user

```
cd backend
npx ts-node src/database/create-admin.ts <email> <password> "<Full Name>"
```

Creates the Super Admin role if absent and assigns it.

13.4 Tests

```
cd backend && npm test          # jest --runInBand
```

Rate limiting is bypassed under `NODE_ENV=test`, and queues use in-memory mocks rather than real Redis.

14. Load testing

A complete **k6 suite already exists** at `k6-load-tests/`, covering every rate-limit tier plus a realistic multi-step workflow test. Read `k6-load-tests/README.md` before writing anything new — it documents the pacing pitfalls in detail.

```
k6 run candidate-rate-limit.js \
  -e BASE_URL=https://<backend-host> \
  -e CANDIDATE_EMAIL=<email> -e CANDIDATE_PASSWORD=<password>
```

If you prefer JMeter or ApacheBench, the same two constraints apply:

1. **Pace below 100 req/min** or the global tier rejects you before you reach any higher tier's boundary (§11).
2. **Set a 60-second request timeout** — a free-tier Render cold start takes 30-60s on the first request and is not a failure.

Seed accounts need `status = 'Active'`. On Render, where `BYPASS_EMAIL_VERIFICATION` is (correctly) unset, register the account then activate it directly:

```
UPDATE "User" SET status = 'Active' WHERE email = '<loadtest account>';
```

Run one script at a time. Several tiers share the global IP-keyed bucket, and the auth (10/15min) and upload (10/10min) budgets are small enough that overlapping runs contaminate each other's results.

15. Routine maintenance

Task	Cadence	Procedure
Check dead-letter queue depth	Weekly	Admin System Health page, or AuditLog where action = 'QUEUE_JOB_FAILED_DLQ'
Orphan asset audit	Monthly	<code>runOrphanAssetCleanup()</code> — read-only; review before deleting anything manually
Database backup before schema change	Every time	Render → database → Backups
Dependency updates	Quarterly	Run inside backend/ or frontend/ respectively, never at the repo root
Disk usage check	Monthly	Render → service → Disks. Growth is unbounded (§12.4)

Secret rotation:

Secret	Blast radius
JWT_ACCESS_SECRET	Logs out all users and invalidates all file-download links (§10.4)
JWT_REFRESH_SECRET	Logs out all users

Secret	Blast radius
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Email stops until updated; verify with <code>npm run email:test</code>
GOOGLE_CLIENT_SECRET	Google login breaks until updated in both Render and Google Cloud Console

Node version is pinned to 22.x. Do not move to 24+ without verifying native modules (bcrypt in particular) build cleanly.

16. Quick reference

Endpoints

Path	Purpose
GET /live	Liveness — process is up
GET /ready	Readiness — process + database
GET /health	Full subsystem JSON
GET /version	Version string
GET /api/v1/api-docs	Swagger UI
/api/v1/auth/*	Authentication
/api/v1/candidates/*	Candidate module
/api/v1/recruiters/*	Recruiter module
/api/v1/admins/*	Admin module (note: plural)
/api/v1/rbac/*	Roles and permissions
/api/v1/company-verification/*	Public, token-authenticated
/files/jfw/:type/:filename	File delivery

Emergency commands

```
# Health
curl https://<backend-host>/health | jq

# Apply migrations (Render Shell)
npx prisma migrate deploy

# Migration status
npx prisma migrate status

# Test email provider
cd backend && npm run email:test you@example.com

# Roll back last commit safely
git revert HEAD && git push origin main

# Activate a stuck account
# psql: UPDATE "User" SET status = 'Active' WHERE email = '...';
```

Escalation checklist

Before escalating, collect: the correlation ID ([CID: ...]) from the failing request, the /health JSON, the Render deploy SHA, and the last 100 log lines around the first error. That set answers most of the follow-up questions immediately.